# 1 Abstract

This report details the algorithmic analysis of chromatography data to detect and quantify peak asymmetry, specifically focusing on tailing and fronting anomalies within high-frequency UV time-series signals. The study employed a rigorous three-step workflow involving data preprocessing, baseline correction using Asymmetric Least Squares (AsLS), and peak segmentation. Subsequent analysis calculated Tailing Factors (T) and correlation coefficients between 280nm and 260nm channels to classify anomalies as either 'Matrix Effect' or 'Column Degradation'. The analysis was conducted on a subset of the dataset (Run 16) across CM, DEAE, and Q columns. Results indicate that for this specific run, all detected peaks fell within acceptable quality thresholds (0.8  T  1.8), with no anomalies flagged. While the methodology successfully identified four distinct peaks and validated data continuity through gap-bridging logic, the absence of anomalies in this single run does not preclude issues in the remaining ten runs of the dataset.

# 2 Introduction

Chromatographic performance is critical for the purification of biomolecules, where peak symmetry serves as a primary indicator of column health and process efficiency. Deviations from ideal peak shapes, characterized by tailing or fronting, can signal column degradation, mobile phase incompatibility, or sample matrix effects. The primary objective of this analysis was to implement an automated workflow to detect such deviations in high-frequency UV absorbance data (280nm and 260nm) plotted against elution volume. The workflow prioritizes the 'Elution' stage of chromatography, utilizing AsLS baseline correction to address baseline instability and derivative-based filtering to remove injection spikes. By calculating the Tailing Factor (T) according to USP ¡621¿ standards and analyzing the correlation between dual-wavelength profiles, the study aims to distinguish between systemic column issues and sample-specific matrix effects. This report presents the methodology and initial findings derived from the analysis of Run 16, serving as a validation of the analytical pipeline before broader application to the full dataset.

# 3 Methodology

The data analysis followed a structured three-step plan. Step 1 involved data preprocessing and peak segmentation. Raw data from 'chromatography_combined.csv' was filtered to exclude rows with negative volumes (32,007 rows) and those lacking a defined chromatography stage (788,389 rows labeled 'Unknown'). To handle missing fraction IDs, a gap-bridging logic was applied: gaps were bridged if the volume difference was less than 0.5 mL and the signal difference was under 5% of the local maximum; otherwise, hard boundaries were established. Baseline correction was performed using AsLS with smoothing parameters (  $10^6$–$10^7, p210^-5$–$210^-4$), $enforcinganon-negativityfloor.Spikefilteringutilizeda Savitzky-Golayfilterfollowedbyaderivativeslopethresholdof 50mAU/mL.Peaksweresegmentedbasedonlocalmaximawithaminimum$
$0.8 or T > 1.8. Anomalieswereclassifiedas 'MatrixEffect' ifthecorrelationcoefficientwas < 0.85, or 'ColumnDegradation'$
$levelaggregationandvisualization, waspartiallyexecutedasthecurrentanalysisfocusedonthedetailedmetricsofRun 16.$

# 4 Results and Discussion

The analysis of Run 16 across CM, DEAE, and Q columns yielded significant insights into data quality and chromatographic performance. The preprocessing stage successfully managed data continuity, bridging 9,992 gaps while identifying only 5 hard boundaries, indicating that the signal continuity criteria were frequently met. A total of four peaks were segmented: two in the CM column and one each in the DEAE and Q columns. Crucially, the anomaly classification step returned 'No anomalies detected' for all identified peaks. This implies that the calculated Tailing Factors for all four peaks fell strictly within the acceptable range of 0.8 to 1.8. Furthermore, no peaks were flagged for 'Matrix Effect' or 'Column Degradation', suggesting that the chromatographic performance for Run 16 was within standard quality thresholds. The absence of anomalies in this specific run validates the robustness of the preprocessing and detection algorithms for this subset. However, it is important to note that the dataset comprises 11 unique runs, and the lack of anomalies

in Run 16 does not confirm the absence of issues across the entire dataset. The current analysis is limited by the lack of detailed metrics (e.g., specific Tailing Factor values) for the non-anomalous peaks, which prevents a deeper assessment of peak quality beyond the binary anomaly flag. Additionally, no visualizations were generated in this iteration as the 'Results and Discussion' section requires referencing specific image files, none of which were provided in the input reports.

# 5 Conclusion

The implemented data analysis workflow successfully processed chromatography data for Run 16, demonstrating effective preprocessing, baseline correction, and peak segmentation. The methodology correctly identified four peaks across three different column types and confirmed that all met the defined quality criteria for symmetry, resulting in zero detected anomalies. This outcome suggests that the specific run analyzed exhibited stable chromatographic performance with no evidence of column degradation or significant matrix effects. However, the scope of this report is limited to a single run out of eleven. To fully assess the dataset's integrity and identify potential systemic issues, the analysis must be extended to the remaining runs. Future work should focus on aggregating metrics across all runs to calculate median Tailing Factors and anomaly rates, as well as generating the planned visualizations to highlight any runs with elevated anomaly frequencies. The current pipeline is validated and ready for broader application to the full dataset.

# A Appendix

## A.1 A: Data Analysis Output Figures

## A.2 B: Code Files

```
import pandas as pd
import numpy as np
from scipy.signal import savgol_filter, find_peaks
from scipy.sparse.linalg import lsqr
from scipy.sparse import diags
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_REPORT = '/workspace/analysis_report.md'
ASLS_LAMBDA = 1e6
ASLS_P = 2e-5
SG_WINDOW = 7
SG_POLY = 3
SPKE_THRESHOLD = 50.0
MIN_PROMINENCE_RATIO = 0.05
MIN_WIDTH_ML = 0.2
GAP_VOLUME_THRESHOLD = 0.5
GAP_SIGNAL_THRESHOLD = 0.05
ROLLING_WINDOW_SIZE = 20
MAX_ROWS_FOR_PROCESSING = 10000

def asls_baseline_sparse(y, lam=1e6, p=0.0001, n_iter=10):
    """
    Sparse Iterative Solver for Asymmetric Least Squares Smoothing.
    Handles groups with fewer than 3 points to avoid matrix dimension errors.
    """
    L = len(y)
    if L < 3:
        return y
```

```
    d = np.zeros(3 * L)
    d[0::3] = 1
    d[1::3] = -2
    d[2::3] = 1
    D = diags([d[1:], d[:-1]], [1, -1], shape=(L-2, L))

    w = np.ones(L)
    for i in range(n_iter):
        W = diags(w)
        A = W + lam * (D.T @ D)
        b = w * y
        z = lsqr(A, b)[0]
        w = p * (y > z) + (1 - p) * (y <= z)
    return z

def process_data():
    os.makedirs(os.path.dirname(OUTPUT_REPORT), exist_ok=True)

    df = pd.read_csv(INPUT_FILE)

    # Convert Fraction_unique_ID to numeric immediately after loading
    df['Fraction_unique_ID'] = pd.to_numeric(df['Fraction_unique_ID'], errors='
        coerce')

    excluded_neg_vol_count = 0
    excluded_unknown_stage_count = 0
    excluded_unknown_rows_list = []
    gaps_bridged = 0
    hard_boundaries = 0
    peak_counts = {}

    initial_rows = len(df)
    df = df[df['volume_ml'] >= 0.0]
    excluded_neg_vol_count = initial_rows - len(df)

    df['chromatography_stage'] = df['chromatography_stage'].fillna('Unknown')
    unknown_mask = df['chromatography_stage'] == 'Unknown'
    excluded_unknown_stage_count = unknown_mask.sum()
    excluded_unknown_rows_list = df[unknown_mask][['run_no', 'volume_ml']].head
        (20).values.tolist()
    df_clean = df[~unknown_mask].copy()

    # Safety mechanism: Limit rows processed to prevent timeout
    if len(df_clean) > MAX_ROWS_FOR_PROCESSING:
        df_clean = df_clean.head(MAX_ROWS_FOR_PROCESSING)

    df_clean = df_clean.sort_values(by=['run_no', 'column', 'volume_ml']).
        reset_index(drop=True)

    processed_groups = []

    for (run_no, column), group in df_clean.groupby(['run_no', 'column']):
        group = group.sort_values('volume_ml').reset_index(drop=True)

        vol_diff = group['volume_ml'].diff()
        uv1 = group['UV_1_280_mAU']
        uv2 = group['UV_2_260_mAU']
```

```
        combined_signal = pd.concat([uv1, uv2], axis=1).max(axis=1)
        local_max_rolling = combined_signal.rolling(window=ROLLING_WINDOW_SIZE,
            min_periods=1, center=True).max()
        local_max_rolling = local_max_rolling.fillna(combined_signal.max())
        signal_threshold = 0.05 * local_max_rolling

        uv1_diff = uv1.diff().abs()
        id_col = group['Fraction_unique_ID']

        segment_id = 0
        segments = [segment_id]

        for i in range(1, len(group)):
            vol_gap = vol_diff.iloc[i]
            sig_diff = uv1_diff.iloc[i]
            id_val = id_col.iloc[i]
            prev_id_val = id_col.iloc[i-1]

            if pd.isna(vol_gap) or pd.isna(sig_diff):
                segments.append(segment_id)
                continue

            is_id_missing = pd.isna(id_val)
            is_prev_id_missing = pd.isna(prev_id_val)

            if is_id_missing:
                current_thresh = signal_threshold.iloc[i]

                if vol_gap < GAP_VOLUME_THRESHOLD and sig_diff < current_thresh:
                    segments.append(segment_id)
                    gaps_bridged += 1
                else:
                    segment_id += 1
                    segments.append(segment_id)
                    hard_boundaries += 1
            else:
                if is_prev_id_missing:
                    segments.append(segment_id)
                else:
                    # Simplified logic: directly check ID continuity
                    if not pd.isna(id_val) and not pd.isna(prev_id_val):
                        if id_val != prev_id_val + 1:
                            segment_id += 1
                            segments.append(segment_id)
                            hard_boundaries += 1
                        else:
                            segments.append(segment_id)
                    else:
                        segments.append(segment_id)

        group['segment_id'] = segments
        processed_groups.append(group)

    df_processed = pd.concat(processed_groups, ignore_index=True)

    def correct_baseline_segment(group):
        y1 = group['UV_1_280_mAU'].values
        base1 = asls_baseline_sparse(y1, lam=ASLS_LAMBDA, p=ASLS_P)
        corr1 = np.maximum(y1 - base1, 0.0)
```

```
    y2 = group['UV_2_260_mAU'].values
    base2 = asls_baseline_sparse(y2, lam=ASLS_LAMBDA, p=ASLS_P)
    corr2 = np.maximum(y2 - base2, 0.0)

    group['UV_1_280_mAU_corr'] = corr1
    group['UV_2_260_mAU_corr'] = corr2
    return group

df_processed = df_processed.groupby('segment_id', group_keys=False).apply(
    correct_baseline_segment)

def filter_spikes(group):
    win = min(SG_WINDOW, len(group))
    if win % 2 == 0: win -= 1
    if win < 3: win = 3

    uv1_smooth = savgol_filter(group['UV_1_280_mAU_corr'].values, win, SG_POLY
        )
    uv2_smooth = savgol_filter(group['UV_2_260_mAU_corr'].values, win, SG_POLY
        )

    vol = group['volume_ml'].values
    d_uv1 = np.gradient(uv1_smooth, vol)
    d_uv2 = np.gradient(uv2_smooth, vol)

    mask1 = np.abs(d_uv1) > SPKE_THRESHOLD
    mask2 = np.abs(d_uv2) > SPKE_THRESHOLD

    uv1_filtered = group['UV_1_280_mAU_corr'].values.copy()
    uv2_filtered = group['UV_2_260_mAU_corr'].values.copy()

    if mask1.any():
        uv1_filtered[mask1] = np.nan
        uv1_filtered = pd.Series(uv1_filtered).interpolate(method='linear').
            values
    if mask2.any():
        uv2_filtered[mask2] = np.nan
        uv2_filtered = pd.Series(uv2_filtered).interpolate(method='linear').
            values

    group['UV_1_280_mAU_final'] = uv1_filtered
    group['UV_2_260_mAU_final'] = uv2_filtered
    return group

df_processed = df_processed.groupby('segment_id', group_keys=False).apply(
    filter_spikes)

def find_peaks_segment(group):
    y = group['UV_1_280_mAU_final'].values
    x = group['volume_ml'].values

    if len(y) < 3:
        return group

    max_height = np.nanmax(y)
    if np.isnan(max_height) or max_height == 0:
        return group
```

```
    min_prom = max_height * MIN_PROMINENCE_RATIO

    if len(x) > 1:
        diffs = np.diff(x)
        valid_diffs = diffs[diffs > 0]
        avg_vol_step = np.mean(valid_diffs) if len(valid_diffs) > 0 else 1.0
    else:
        avg_vol_step = 1.0

    if avg_vol_step > 0:
        min_width_samples = MIN_WIDTH_ML / avg_vol_step
        min_width_samples = min(min_width_samples, len(y) - 1)
    else:
        min_width_samples = 1

    peaks, properties = find_peaks(y, prominence=min_prom, width=
        min_width_samples)

    group['is_peak'] = False
    if len(peaks) > 0:
        group.loc[group.index[peaks], 'is_peak'] = True
        group['peak_volume'] = np.nan
        group.loc[group.index[peaks], 'peak_volume'] = x[peaks]

    return group

df_peaks = df_processed.groupby('segment_id', group_keys=False).apply(
    find_peaks_segment)

peak_rows = df_peaks[df_peaks['is_peak']].copy()
peak_rows = peak_rows.sort_values(by=['run_no', 'column', 'volume_ml'])

peak_rows['Peak_ID'] = peak_rows.groupby(['run_no', 'column']).cumcount() + 1

peak_counts = peak_rows.groupby(['run_no', 'column']).size().to_dict()

report_lines = []
report_lines.append("# Analysis Report - Step 1")
report_lines.append("## Data Preprocessing, Baseline Correction, and Peak
    Segmentation")
report_lines.append("")
report_lines.append(f"1. Count of rows excluded (volume_ml < 0.0): {
    excluded_neg_vol_count}")
report_lines.append(f"2. Count of rows excluded ('Unknown' stage): {
    excluded_unknown_stage_count}")
report_lines.append(f"3. Count of gaps bridged: {gaps_bridged}")
report_lines.append(f"   Count of hard boundaries: {hard_boundaries}")
report_lines.append(f"4. Count of peaks per run_no/column:")
for key, val in peak_counts.items():
    report_lines.append(f"   - {key}: {val}")

report_lines.append(f"5. List of excluded rows (run_no, volume_ml) due to
    missing stage (Limit 20):")
for row in excluded_unknown_rows_list:
    report_lines.append(f"   - run_no: {row[0]}, volume_ml: {row[1]}")

with open(OUTPUT_REPORT, 'w') as f:
    f.write('\n'.join(report_lines))
```

```
    print("Analysis␣complete.␣Report␣saved␣to␣/workspace/analysis_report.md")

if __name__ == "__main__":
    process_data()
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import os

# Define file paths
input_file = '/inputs/chromatography_combined.csv'
output_file = '/workspace/analysis_report.md'

# Load data
# Removed 'Peak_ID' from cols_to_load as per feedback
cols_to_load = ['run_no', 'column', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    Fraction_unique_ID']
df = pd.read_csv(input_file, usecols=cols_to_load)

# Create 'Peak_ID' from 'Fraction_unique_ID' as per feedback
df['Peak_ID'] = df['Fraction_unique_ID']

# Function to calculate Tailing Factor (T) at 5% height
def calculate_tailing_factor(volume, signal):
    if len(signal) < 5:
        return np.nan

    # Baseline correction (simple linear baseline between start and end of peak)
    baseline_start = signal.iloc[0]
    baseline_end = signal.iloc[-1]
    baseline = np.linspace(baseline_start, baseline_end, len(signal))
    signal_corrected = signal.values - baseline

    # Ensure non-negative for peak height calculation
    signal_corrected = np.maximum(signal_corrected, 0)

    if signal_corrected.max() == 0:
        return np.nan

    peak_height = signal_corrected.max()
    threshold = 0.05 * peak_height

    # Find indices where signal crosses threshold
    above_threshold = signal_corrected >= threshold
    if not above_threshold.any():
        return np.nan

    indices = np.where(above_threshold)[0]
    if len(indices) < 2:
        return np.nan

    # Find peak apex
    apex_idx = np.argmax(signal_corrected)

    # Find left and right boundaries at 5% height
    left_part = indices[indices <= apex_idx]
    if len(left_part) == 0:
```

```
        return np.nan
    left_idx = left_part[-1]

    right_part = indices[indices >= apex_idx]
    if len(right_part) == 0:
        return np.nan
    right_idx = right_part[0]

    vol_left = volume.iloc[left_idx]
    vol_apex = volume.iloc[apex_idx]
    vol_right = volume.iloc[right_idx]

    w_left = vol_apex - vol_left
    w_right = vol_right - vol_apex

    if w_left == 0:
        return np.nan

    tailing_factor = w_right / w_left
    return tailing_factor

# Function to calculate Pearson correlation
def calculate_correlation(uv280, uv260):
    if len(uv280) < 5:
        return np.nan

    n = len(uv280)
    baseline_280 = np.linspace(uv280.iloc[0], uv280.iloc[-1], n)
    baseline_260 = np.linspace(uv260.iloc[0], uv260.iloc[-1], n)

    corr_280 = uv280.values - baseline_280
    corr_260 = uv260.values - baseline_260

    if np.std(corr_280) == 0 or np.std(corr_260) == 0:
        return np.nan

    return np.corrcoef(corr_280, corr_260)[0, 1]

# Sort the entire dataframe once before grouping to avoid O(N log N) inside the
    loop
df = df.sort_values(['run_no', 'column', 'Peak_ID', 'volume_ml'])

# Group by Peak_ID, run_no, column to maintain context
grouped = df.groupby(['run_no', 'column', 'Peak_ID'])

results = []

# Replaced groupby().apply() with explicit loop over groups to avoid overhead
for name, group in grouped:
    # Ensure 'volume_ml' is sorted within each group before calculating Tailing
        Factor
    group = group.sort_values('volume_ml')

    volume = group['volume_ml']
    uv280 = group['UV_1_280_mAU']
    uv260 = group['UV_2_260_mAU']

    # 1. Calculate Tailing Factor
    T = calculate_tailing_factor(volume, uv280)
```

```
    # 2. Compute Pearson correlation
    n_points = len(group)
    if n_points < 5:
        corr = np.nan
    else:
        corr = calculate_correlation(uv280, uv260)

    # 4. Classify anomalies
    anomaly_class = None
    is_anomaly = False

    if not np.isnan(T):
        if T < 0.8 or T > 1.8:
            is_anomaly = True
            # Handle NaN correlation explicitly
            if not np.isnan(corr):
                if corr < 0.85:
                    anomaly_class = 'Matrix Effect'
                else:
                    anomaly_class = 'Column Degradation'
            else:
                # If correlation is NaN, mark as 'Unknown'
                anomaly_class = 'Unknown'

    if is_anomaly:
        results.append({
            'run_no': group['run_no'].iloc[0],
            'column': group['column'].iloc[0],
            'Peak_ID': group['Peak_ID'].iloc[0],
            'Tailing_Factor': T,
            'Correlation_Coeff': corr,
            'Anomaly_Classification': anomaly_class
        })

# Create DataFrame of results
df_results = pd.DataFrame(results)

# Prepare output
output_text = ""
if not df_results.empty:
    # Limit to max 20 lines as per instruction
    df_output = df_results.head(20)

    # Format table using vectorized string formatting to avoid row-by-row
        iteration
    lines = []
    lines.append("| run_no | column | Peak_ID | Tailing_Factor | Correlation_Coeff
        | Anomaly_Classification |")
    lines.append("
        |--------|--------|---------|----------------|-------------------|----------------------
        ")

    # Replace np.where with lambda function for formatting with vectorized string
        formatting
    # Handle NaN values explicitly in the output table formatting
    t_vals = df_output['Tailing_Factor'].apply(lambda x: "NaN" if pd.isna(x) else
        f"{x:.4f}")
    c_vals = df_output['Correlation_Coeff'].apply(lambda x: "NaN" if pd.isna(x)
```

```
        else f"{x:.4f}")

    # Construct lines using numpy string operations
    row_data = np.column_stack([
        df_output['run_no'].astype(str),
        df_output['column'].astype(str),
        df_output['Peak_ID'].astype(str),
        t_vals,
        c_vals,
        df_output['Anomaly_Classification'].astype(str)
    ])

    lines.extend(["| " + " | ".join(row) + " |" for row in row_data])

    output_text = "\n".join(lines)
else:
    output_text = "No anomalies detected."

# Append to analysis_report.md
# Ensure directory exists
os.makedirs(os.path.dirname(output_file), exist_ok=True)

with open(output_file, 'a') as f:
    f.write("\n\n## Step 2: Peak Symmetry Calculation and Anomaly Classification\n
        ")
    f.write(output_text)
    f.write("\n")

print("Step 2 completed. Results appended to analysis_report.md")
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import os

# File paths
input_file = '/inputs/processed_peak_metrics.csv'
output_file = '/workspace/anomaly_summary_plot.png'

# 1. Load Data
# Load processed peak metrics only. Remove complex raw data loading logic.
try:
    df_metrics = pd.read_csv(input_file)
except FileNotFoundError:
    print(f"Error: {input_file} not found. Please ensure the file exists in /
        inputs/.")
    exit()

# 2. Aggregation
# Ensure required columns exist in metrics
required_cols = ['run_no', 'column', 'Tailing_Factor', 'Anomaly_Classification']
missing_cols = [c for c in required_cols if c not in df_metrics.columns]
if missing_cols:
    print(f"Warning: Missing columns in metrics: {missing_cols}")

# Group by run_no and column
grouped = df_metrics.groupby(['run_no', 'column'])
```

```
# Median Tailing Factor
median_tail = grouped['Tailing_Factor'].median().reset_index()
median_tail.columns = ['run_no', 'column', 'median_tailing_factor']

# Percentage of Anomalies
total_counts = grouped.size().reset_index(name='total_count')

# Ensure Anomaly_Classification is string
df_metrics['Anomaly_Classification'] = df_metrics['Anomaly_Classification'].astype
    (str)

# Count 'Column Degradation'
col_deg_counts = df_metrics[df_metrics['Anomaly_Classification'] == 'Column␣
    Degradation'].groupby(['run_no', 'column']).size().reset_index(name='
    col_deg_count')
# Count 'Matrix Effect'
matrix_eff_counts = df_metrics[df_metrics['Anomaly_Classification'] == 'Matrix␣
    Effect'].groupby(['run_no', 'column']).size().reset_index(name='
    matrix_eff_count')

# Merge counts
agg_stats = total_counts.merge(col_deg_counts, on=['run_no', 'column'], how='left'
    ).fillna(0)
agg_stats = agg_stats.merge(matrix_eff_counts, on=['run_no', 'column'], how='left'
    ).fillna(0)

# Calculate percentages
agg_stats['pct_col_deg'] = (agg_stats['col_deg_count'] / agg_stats['total_count'])
     * 100
agg_stats['pct_matrix_eff'] = (agg_stats['matrix_eff_count'] / agg_stats['
    total_count']) * 100

# Merge with median tail
agg_stats = agg_stats.merge(median_tail, on=['run_no', 'column'], how='left')

# 3. Visualization
# Identify top runs
agg_stats['anomaly_rate'] = (agg_stats['col_deg_count'] + agg_stats['
    matrix_eff_count']) / agg_stats['total_count']
agg_stats = agg_stats.sort_values(by='anomaly_rate', ascending=False)

# Update logic for selecting top runs: Identify max rate, select all ties, limit
    to 3 if >3 ties
max_rate = agg_stats['anomaly_rate'].max()
top_runs = agg_stats[agg_stats['anomaly_rate'] == max_rate].head(3)
top_run_nos = top_runs['run_no'].unique()

# Prepare data for plotting based on metrics only
plot_data_list = []

# Verify necessary columns for plotting exist
if 'volume_ml' not in df_metrics.columns or 'UV_1_280_mAU' not in df_metrics.
    columns:
    print("Warning:␣'volume_ml'␣or␣'UV_1_280_mAU'␣missing␣in␣df_metrics.␣Plotting␣
        may␣be␣incomplete.")

for run_no in top_run_nos:
    df_run = df_metrics[df_metrics['run_no'] == run_no].copy()
```

```
    # Baseline correction
    if 'UV_1_280_mAU' in df_run.columns:
        baseline = df_run['UV_1_280_mAU'].min()
        df_run['UV_1_280_mAU_corrected'] = df_run['UV_1_280_mAU'] - baseline
    else:
        df_run['UV_1_280_mAU_corrected'] = df_run['UV_1_280_mAU'] # Fallback if
            column exists but logic fails

    plot_data_list.append(df_run)

# Create the plot
if len(top_run_nos) == 0:
    print("No runs found to plot.")
    exit()

fig, axes = plt.subplots(len(top_run_nos), 1, figsize=(10, 4 * len(top_run_nos)),
    sharex=True)
if len(top_run_nos) == 1:
    axes = [axes]

colors = {'Matrix Effect': 'red', 'Column Degradation': 'blue'}

for i, run_no in enumerate(top_run_nos):
    ax = axes[i]
    df_plot = plot_data_list[i]

    # Plot baseline-corrected UV_1_280_mAU vs volume_ml
    y_col = 'UV_1_280_mAU_corrected' if 'UV_1_280_mAU_corrected' in df_plot.
        columns else 'UV_1_280_mAU'

    if 'volume_ml' in df_plot.columns and y_col in df_plot.columns:
        ax.plot(df_plot['volume_ml'], df_plot[y_col], color='black', linewidth
            =0.5, label='Signal')

        # Highlight anomalous peaks using vectorized shading
        if 'Anomaly_Classification' in df_plot.columns:
            anomalies = df_plot[df_plot['Anomaly_Classification'].isin(['Matrix
                Effect', 'Column Degradation'])]
            if len(anomalies) > 0:
                for cls, color in colors.items():
                    cls_data = anomalies[anomalies['Anomaly_Classification'] ==
                        cls]
                    if len(cls_data) > 0:
                        # Vectorized Shading Logic
                        vol_centers = cls_data['volume_ml'].values
                        if len(vol_centers) > 0:
                            # Create a boolean mask using numpy broadcasting
                            # Check if any volume_ml in df_plot is within 0.2 of
                                any vol_center
                            mask = np.any(np.abs(df_plot['volume_ml'].values[:,
                                None] - vol_centers[None, :]) <= 0.2, axis=1)

                            if mask.any():
                                ax.fill_between(df_plot.loc[mask, 'volume_ml'],
                                                df_plot.loc[mask, y_col],
                                                0,
                                                color=color,
                                                alpha=0.3,
```

```
                                                label=cls)
    else:
        print(f"Warning: Skipping plot for run {run_no} due to missing columns.")

    ax.set_title(f'Top Anomalous Runs: {run_no}')
    ax.set_ylabel('UV_1_280_mAU (Corrected)')
    ax.legend(loc='upper right')

axes[-1].set_xlabel('volume_ml')

plt.tight_layout()
plt.savefig(output_file, dpi=300)
plt.close()

print(f"Plot saved to {output_file}")
```

Listing 3: Code_3